DATAMENTORS ACADEMY / STUDENT GUIDE

Your two-week prework

Build a small shared starting point in Python, Git, robotics and ROS 2 Humble - then bring your questions to the first Academy sessions.

Self-paced | About two weeks | No robot required | Personal GitHub account

Start here

Work through this guide at your own pace over roughly two weeks. The ten sessions below are a suggested order, not a daily attendance requirement. A typical session takes about one to three hours. You are **not** expected to complete any whole online course or earn a certificate. There are two small programming deliverables: a Python episode-review CLI and a C++ ROS 2 state monitor. Show what you made and identify questions to resolve together at the start of the Academy; this is not graded. Python and Git prepare week 1, recording quality and ROS 2 topics prepare week 2, and the C++ node prepares week 3.

You should already know the basics of one programming language. If Python is new to you, use the linked introductory notes first and record where you need help; the Academy Lead will use that information to support you, not to exclude you.

> **No access to Datamentors' corporate GitHub is needed or provided.** Create your own free GitHub account and use a private repository for this prework. Use your own computer and the synthetic files in this package. Do not use company data, a physical robot, or an unapproved model checkpoint. Linear, an Academy practice repository and PR review are introduced during Academy week 1.

Files in this package

- `sample-episodes.csv` - synthetic SO-101 pickup-to-bin episodes
- `ros2-graph-case.txt` - synthetic graph and timing snapshot for the same cell
- `ros2_ws/src/academy_cell_monitor/` - C++ ROS 2 starter and tests
- `compose.yaml` - local ROS 2 Humble environment

Unzip the whole package before starting.

Install only what you need

- Python 3.10 or newer, Git, and a text editor.
- Docker with Compose and a running Docker engine to build and run the supplied ROS 2 Humble workspace; native Humble also works.
- Internet for the linked readings, Docker image download and optional GitHub delivery. No robot, camera or GPU is needed.

Check `python3 --version`, `git --version`, `docker compose version` and `docker info`. If setup fails, submit the C++ source and document the blocker; we will run it together in week 1.

Suggested ten-session plan

Session	Theme	Do this	Assigned course or reading
1	Orientation and setup	Unzip the whole package. Create a personal GitHub account if needed. Check Python, Git and Docker availability; record any setup blocker.	CS50P Week 0: functions and variables - selected notes GitHub: create a personal account
2	Python collections	Practice functions, loops, lists and dictionaries. Sketch how you would count episodes by capture quality and session.	CS50P Week 2: loops, lists and dictionaries - selected notes
3	Python file I/O and arguments	Read `sample-episodes.csv` with Python's standard `csv` module. Accept the CSV path as one positional command-line argument. Start the mini-project below.	CS50P Week 6: file I/O and CSV - selected notes Python: argparse positional arguments
4	Testing and errors	Finish the summary program. Add at least two tests or documented self-checks; handle a missing input file clearly. Automated tests may use the standard-library `unittest` module; `pytest` is not required.	CS50P Week 3: exceptions - selected notes Python: unittest basic example, if writing automated tests
5	Shell and Git	Navigate the project folder, inspect files, initialise a local Git repository, create a feature branch and commit your work.	Missing Semester: introduction to the shell Missing Semester: version control and Git
6	Robotics concepts	Draw the SO-101 leader/follower arms, camera, cube and bin. Explain observation, command, joint state, actuator limit and emergency stop.	Hugging Face Robotics Course: classical robotics, introductory overview only
7	ROS 2 Humble concepts	Read the node/topic introductions and answer the graph questions below. Start the supplied Humble container and build the C++ starter if your machine allows it.	ROS 2 Humble: understanding nodes ROS 2 Humble: understanding topics
8	C++ ROS 2 implementation	Complete `is_stale()` and the node's `on_state()` subscription callback. Test no message, recent state and old state.	LearnCpp: functions and C++17 basics - selected notes ROS 2 Humble: C++ publisher/subscriber tutorial
9	ROS 2 test and observation	Build and test the package. Run the monitor, publish synthetic joint states, stop publishing and observe `/cell/state_fresh` switch from false to true to false. If setup fails, document your attempt and explain the intended logic.	ROS 2 Humble: topic CLI tools Docker: container basics if needed
10	Explain and share	Write a short README with commands, results, design choices, blockers and questions for both programs. Run your checks again. Create a private repository in your personal GitHub account and push your practice branch.	GitHub: create a repository GitHub: publish an existing local repository

How much to read: use the named sections for the day's task. The linked courses contain far more material than this prework requires. In the robotics reading, stop before detailed kinematics mathematics; that belongs in the Academy.

One small mini-project: review a recording table

The CSV is fabricated for this exercise: it contains no real recordings or sensor messages. It represents one fixed-object SO-101 pickup-to-bin task across two sessions and is not an accepted production dataset. `capture_quality` is `usable` or `unusable` for review; `task_outcome` is `success`, `failure` or `unknown`. These are different questions: E005 is a usable recording of a failed attempt. Do not silently relabel a failed task as a successful one or treat a non-empty recording as approved for training.

1. Create `analyze_episodes.py`. Take the path to a CSV file as a command-line argument.
2. Print total, usable and unusable counts; print usable counts by session. Separately count successful and failed tasks among usable episodes.
3. Flag suspicious rows, for example a `usable` row with zero frames or non-positive duration, or a quality or outcome value outside the allowed values above. Do not silently delete rows.
4. Add at least two tests or self-check cases. Explain how to run them in your README.

> **Self-check on the supplied CSV:** 12 episodes: 9 usable and 3 unusable. Usable episodes: 5 in session A and 4 in session B. Among the 9 usable episodes, 8 tasks succeeded and 1 failed. There are no contradictory metadata rows in the supplied file; create one extra bad row in a separate test file to prove your warning works.

Use the Python standard library only. A concise, working program is more useful than a large framework. If you use an AI assistant, verify the result yourself and note in the README which parts were assisted.

C++ and ROS 2: monitor the same cell

Read `ros2-graph-case.txt` and the starter package's `README.md`. The graph and timing snapshot are invented on paper, not running ROS nodes or recorded robot data. A node is a running component; a topic carries a stream of messages. The leader arm's measured joints are observations, while the follower command is an intended action. Neither proves that the follower reached the requested state. The C++ node listens only to measured follower joint states and publishes a diagnostic; it never commands the robot.

1. Draw arrows from publishers to subscribers for the five topics in the case. Identify who creates commands, reports measured state, records data and publishes `/cell/state_fresh`.
2. At the supplied check time, calculate the age of the follower joint state. Explain why `/cell/state_fresh` must be false and why command publication alone does not prove movement.
3. Complete `is_stale()` in `freshness.hpp` and record local arrival time in the node's `on_state()` callback. Build and run the supplied tests. The starter compiles, but the test intentionally fails until you implement the logic.
4. Use the synthetic ROS 2 publisher in the package README to demonstrate the false-true-false transition. Name two physical safety limits that would matter before moving a real arm; this diagnostic node is not a safety controller.

> **Where the live messages come from:** the `ros2 topic pub` command in the starter README creates example `JointState` messages locally at 5 Hz. It supplies a fixed `joint1` position of `0.0`; it does not read a robot or replay the graph case. Use `compose.yaml` to run Humble in one local container. If Docker/native Humble fails, submit your C++ change and a clear setup explanation; bring the blocker to week 1.

Practice a local engineering workflow

Use Git locally, then publish both programming exercises in one private repository under your personal GitHub account. Keep only the supplied synthetic files and your own work in this folder. The included `.gitignore` excludes generated ROS 2 build outputs. Replace the example name and email with your own before committing:

```
git init
git switch -c feature/prework-cell-exercises
git config user.name "Your Name"
git config user.email "your-email@example.com"
git status
git add .
git status
git commit -m "Add prework cell exercises"
git log --oneline
```

Create an **empty, private** personal GitHub repository without starter files. From your practice folder, replace `<repository-url>` with its URL:

```
git remote add origin <repository-url>
git push -u origin feature/prework-cell-exercises
```

Keep it private. If a reviewer account is supplied, grant it access; otherwise send a ZIP of your work through the same channel that delivered this pack. A ZIP is also fine if GitHub setup fails. Do not use company Linear or corporate GitHub yet. The Academy team will guide the first reviewed PR in week 1. Never upload credentials, real recordings or company material.

What to share and discuss

Before the first day, reply through the same channel that delivered this pack with your private repository link (and reviewer access if an account was supplied), or attach a ZIP of your work. Bring two or three questions and briefly explain what worked and what did not. You will discuss your work with the Academy Lead and peers in week 1; there is no score or pass/fail decision.

- ☐ `analyze_episodes.py`, the sample CSV, and at least two tests or self-checks.
- ☐ A short `README.md` with run instructions, expected summary, setup problems, questions and a meaningful Git commit if Git worked.
- ☐ Your changed C++ `freshness.hpp` and `cell_monitor.cpp`, test result or setup blocker, and a short explanation of the five-topic graph and false-true-false transition.